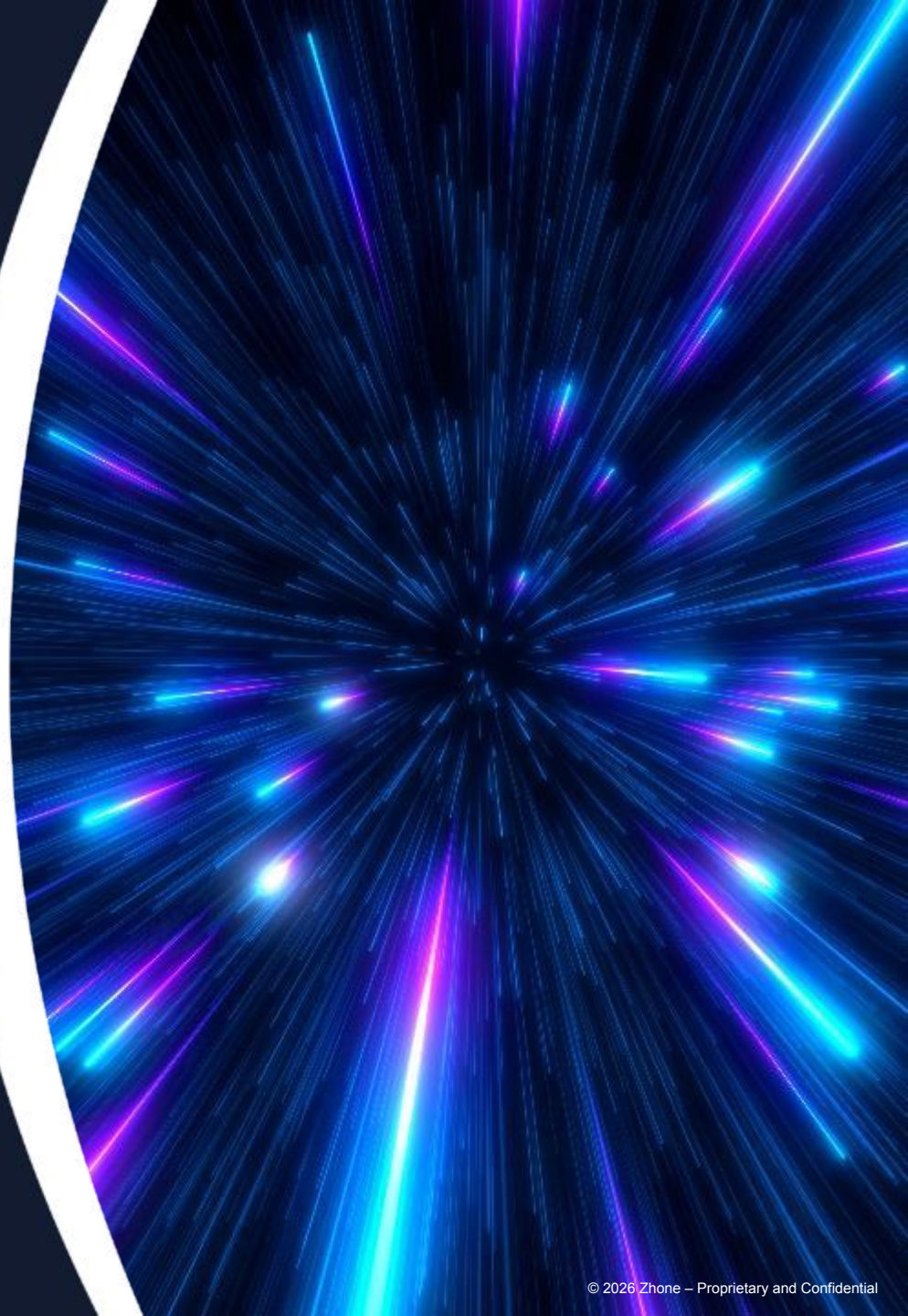
zhone

Embedded Linux Overview, Cross-compilation, Boot Process

Presenter: Nguyen Manh Hung
Date: 29/10/2026

Embedded Trainees



© 2026 Zhone – Proprietary and Confidential

PROBLEM STATEMENT



Linux Computer



Raspberry pi 3 Model B

zhone
© 2026 Zhone – Proprietary and Confidential

OVERVIEW

1

Embedded Linux Overview

- What is Embedded Linux?
- Components of an embedded Linux system.

2

Cross-compilation

- What is cross-compilation?
- Why we need it?
- Detail build the cross-compilation.
- What does a toolchain consist of?
- Choosing a C library: glibc, uClibc-ng, or musl?

3

Boot Process

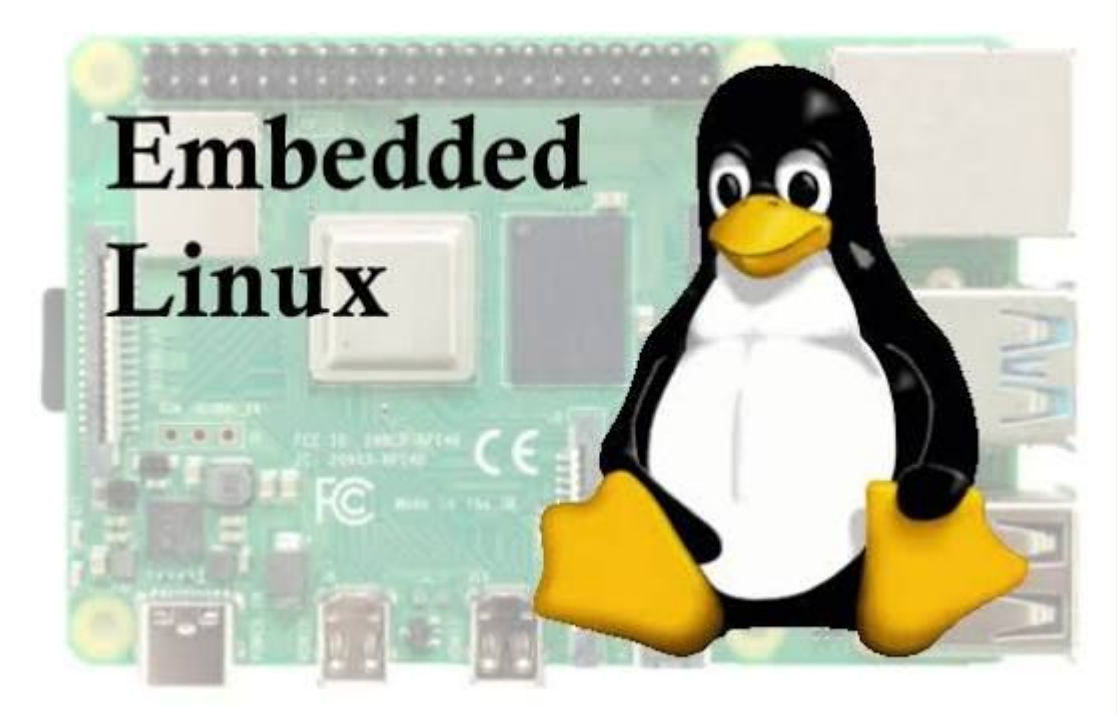
- The entire startup process.
- Linux kernel, Device Tree and root filesystem.
- What does U-Boot do?

zhone
© 2026 Zhone – Proprietary and Confidential

01

Embedded Linux Overview

- What is Embedded Linux?
- Components of an embedded Linux system.



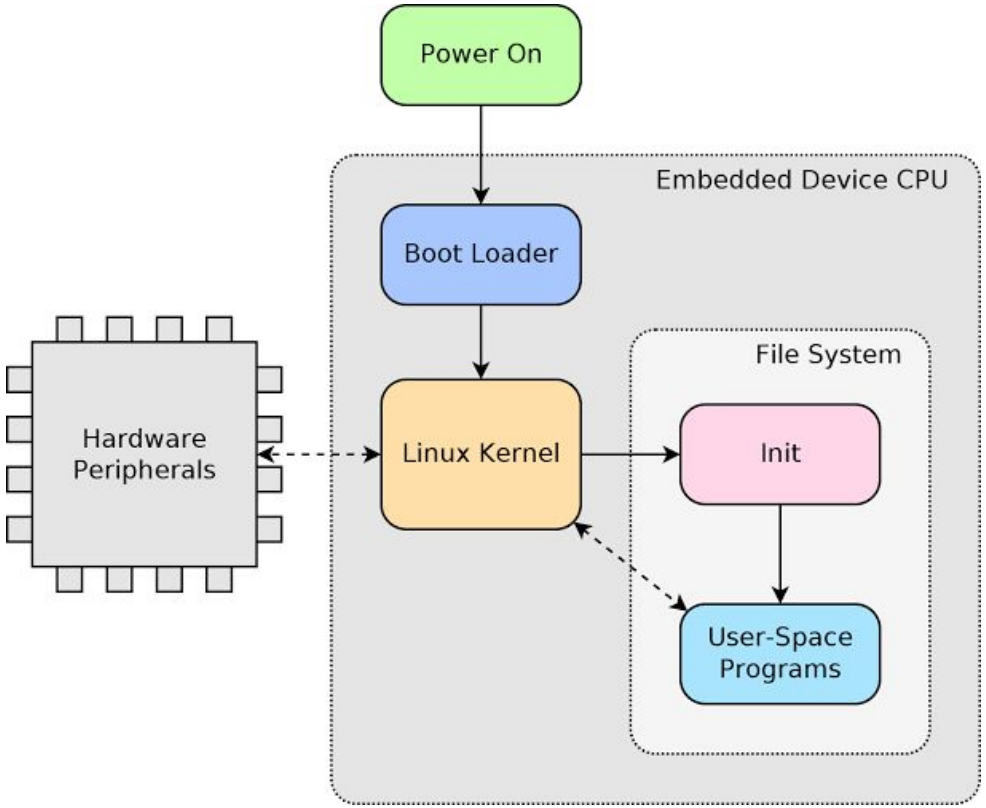
zhone
© 2026 Zhone – Proprietary and Confidential

1. Embedded Linux Overview

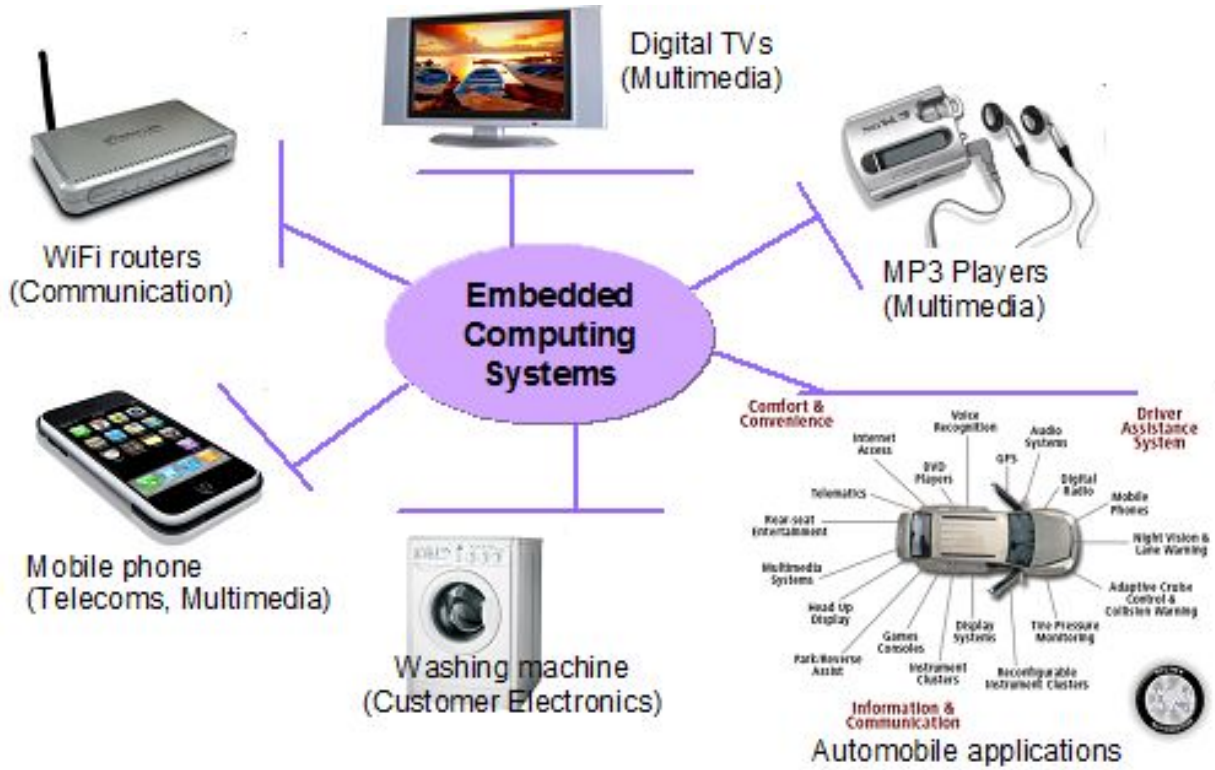
What is Embedded Linux?

Embedded Linux is a version of the Linux operating system that has been customized and streamlined to run on embedded systems.

Components of an embedded Linux system.



Common Practical Applications:

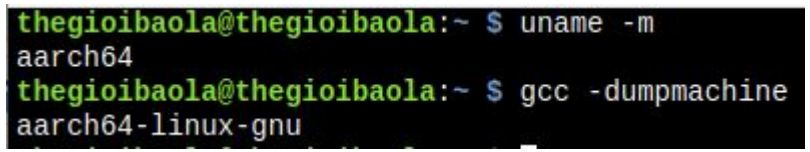


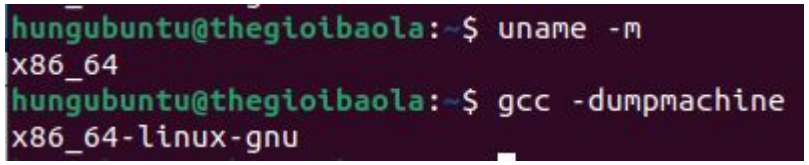
zhone
© 2026 Zhone – Proprietary and Confidential

1. Embedded Linux Overview

How can I use an Ubuntu machine to create software that will run on a Raspberry pi?

Characteristic	Ubuntu PC (Host)	Raspberry pi 3 B (Target)
Device	Ubuntu computer	Raspberry Pi 3 Model B
CPU	x86-64	ARM Cortex-A53
Software architecture	x86-64	AArch64
Role	Compiling software	Running software
Operating system	Ubuntu 22.04	Linux for ARM64





zhone
© 2026 Zhone – Proprietary and Confidential

02

Cross-compilation

Cross-compilation: How can an x86-64 Ubuntu machine build programs that run on an ARM64 Raspberry Pi?



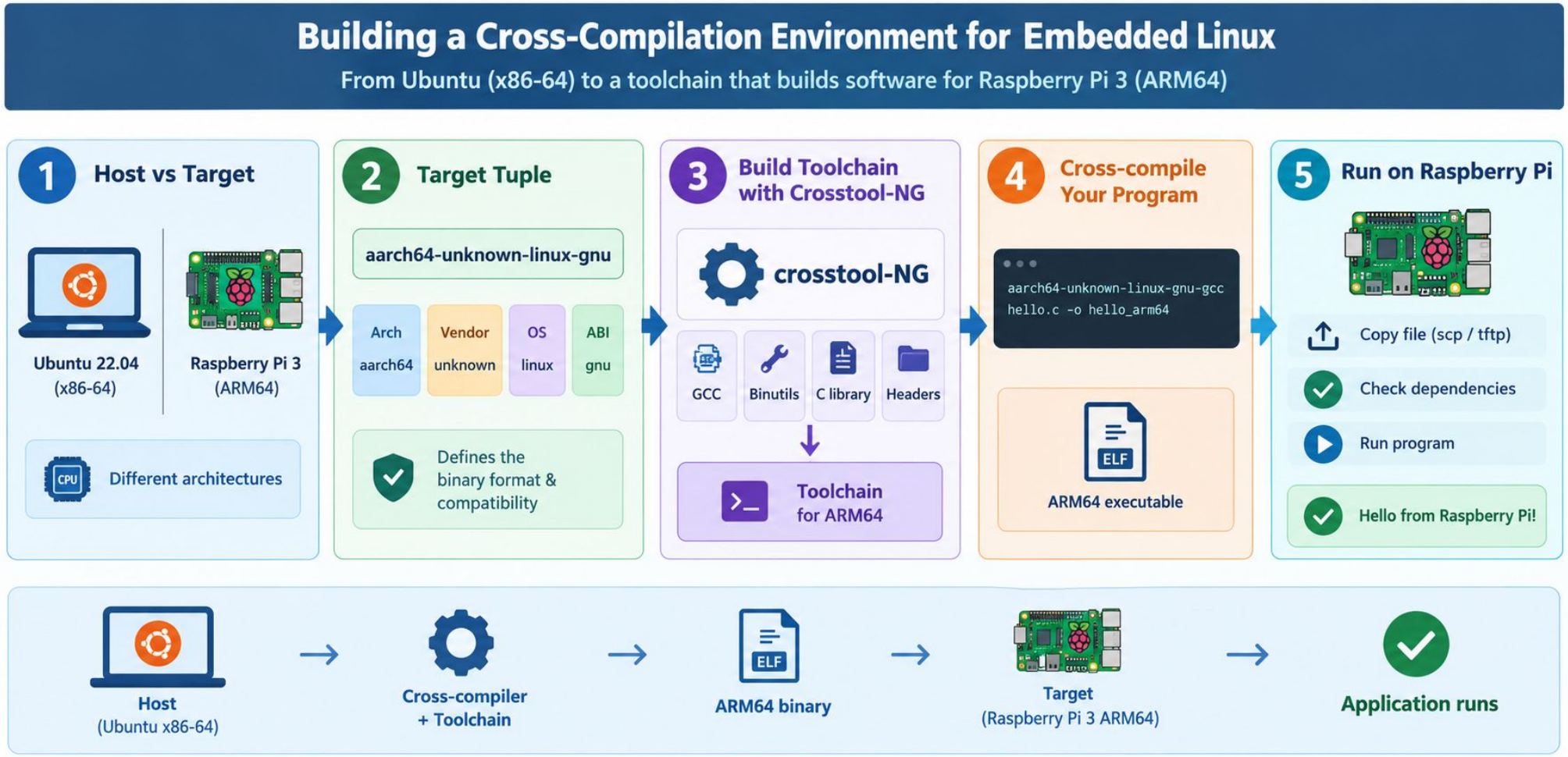
- What is cross-compilation?
- Why we need it?
- Detail build the cross-compilation.
- What does a toolchain consist of?
- Choosing a C library: glibc, uClibc-ng, or musl?

zhone
© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

2.1 What is cross-compilation?

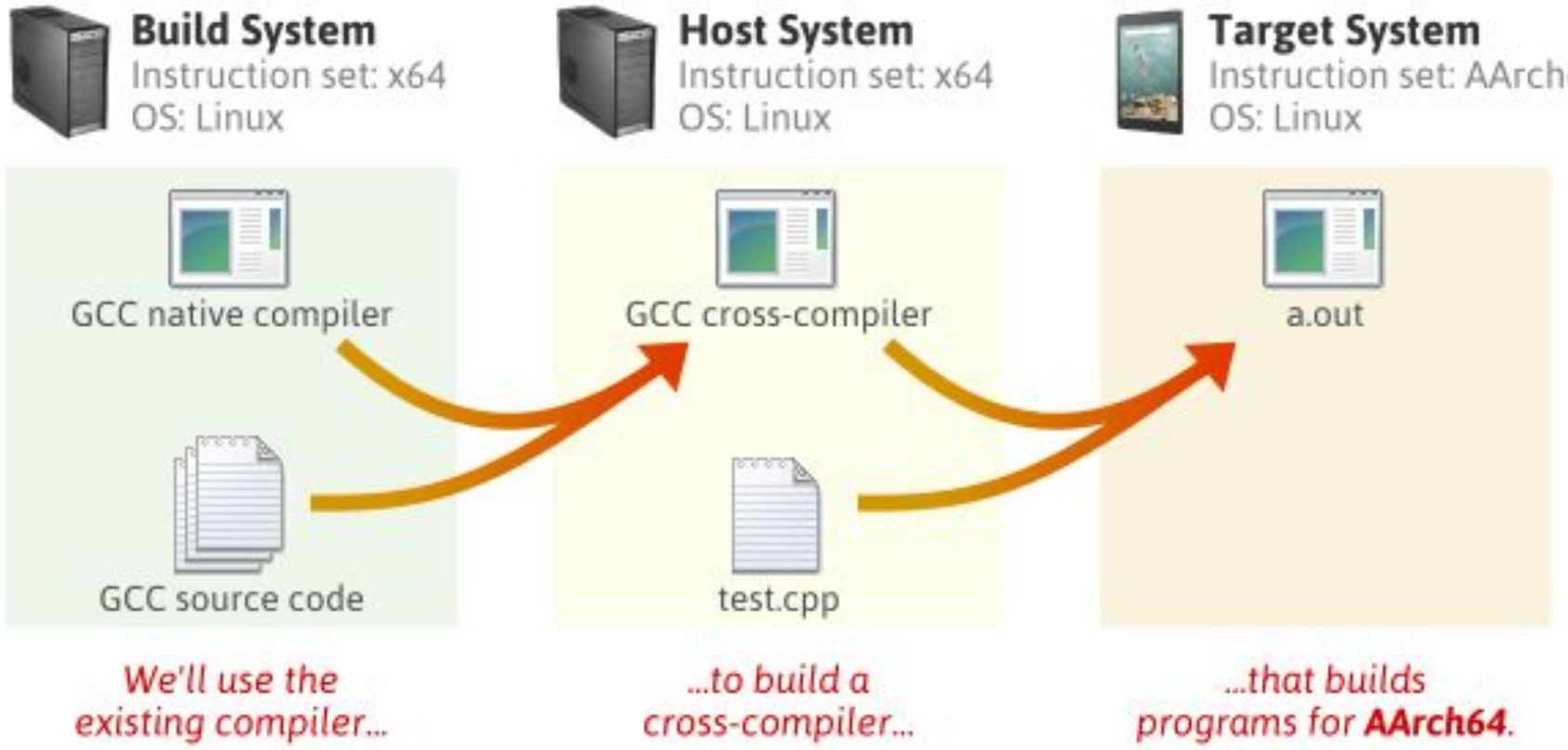
Cross-compilation is the process of compiling software on one system to produce executable code for a different target system.



© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

2.2 build - host - target



Dumping internal crosstool-NG configuration
Building a toolchain for:
build = x86_64-pc-linux-gnu
host = x86_64-pc-linux-gnu
target = aarch64-unknown-linux-gnu

zhone
© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

2.3 target tuple: (*<arch>-<vendor>-<os>-<abi>*)

Component	Meaning	Common Value
Architecture	64-bit ARM CPU	arm, aarch64, riscv64, mips, x86_64
Vendor	No specific vendor specified	none, unknown, pc, poky, buildroot
OS	Linux operating system	linux, none (bare-metal), w64
ABI/environment	GNU/Linux environment, typically associated with glibc	gnu, gnueabihf (hard-float), eabi, musl,

Example: *aarch64-unknown-linux-gnu*

zhone
© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

2.4 Choosing a cross-compilation setup approach:

a. Use a ready-to-use cross-compiling toolchains

Toolchain	Target architecture	Example use case
aarch64-linux-gnu-gcc	ARM64 / AArch64	Raspberry Pi running 64-bit Linux
arm-linux-gnueabihf-gcc	ARM 32-bit, hard-float	Raspberry Pi running 32-bit Linux
arm-linux-gnueabi-gcc	ARM 32-bit, soft-float ABI	Older ARM Linux systems
arm-none-eabi-gcc	ARM bare-metal	STM32, firmware not running Linux

Advantages:

- ➢ Quick to set up.
- ➢ No need to manually build GCC, binutils, or the C library.
- ➢ Well-suited for learning how to cross-compile C applications.

Disadvantages:

- ➢ Difficult to maintain full control over the versions and configurations of individual components.
- ➢ The toolchain may be incompatible with the runtime environment of a specific target if not carefully verified.
- ➢ Not an optimal choice if the project requires a custom, precisely reproducible build environment.

zhone
© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

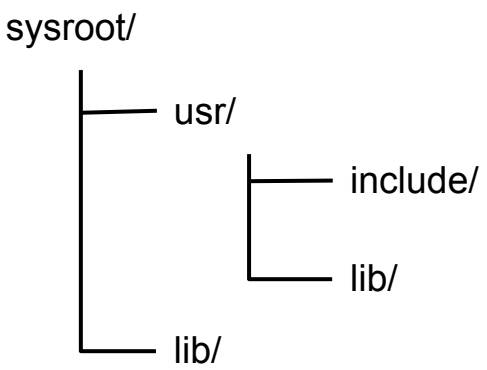
b. Building a custom toolchain using Crosstool-NG

What is Crosstool-NG?

Crosstool-NG is a toolchain generator that automates the process of building cross-compilation toolchains.

Components of a toolchain:

- **GCC** — Compiler
- **Binutils** — Tools for processing machine code
- **C library** — API environment for applications
- **Sysroot** — Target development environment



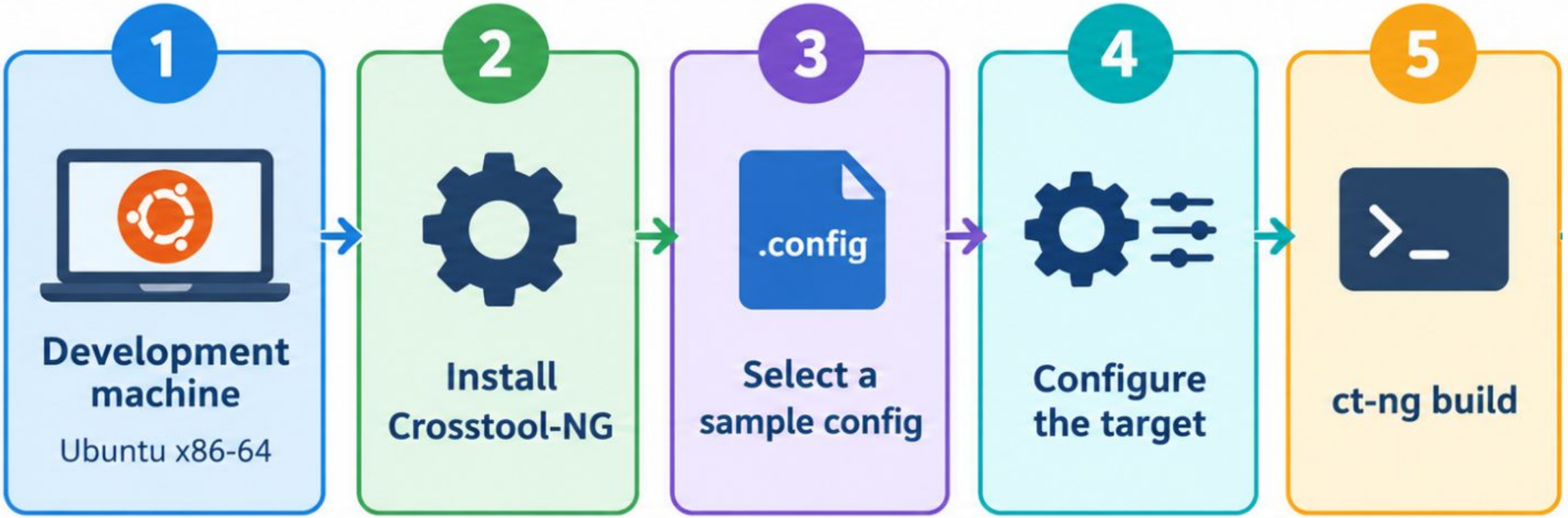
Common options:

- ➔ **glibc**: suitable when the GNU/Linux ecosystem and broad compatibility are required.
- ➔ **musl**: a lightweight C library, typically used in systems designed for minimalism.
- ➔ **uClibc-ng**: an option geared toward embedded environments with specific requirements.

zhone
© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

b. Building a custom toolchain using Crosstool-NG



Result cross-compilation toolchain: *aarch64-unknown-linux-gnu*

zhone
© 2026 Zhone – Proprietary and Confidential

2. Cross-compilation

2.5 Run on Raspberry pi 3 B

Run example code:

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, Raspberry Pi! \n");
    return 0;
}
```

Compile program to ARM:

aarch64-unknown-linux-gnu-gcc hello.c -o hello_arm64

Run on computer ubuntu:

```
hungubuntu@thegioibaola:~/Vim_C_code/Embedded_Training/session_10$ ./hello_arm64
bash: ./hello_arm64: cannot execute binary file: Exec format error
```

Run on Raspberry pi 3 B:

```
thegioibaola@thegioibaola:/mnt/usb/thegioibaola/code_c $ ./hello_arm64
Hello, Raspberry Pi!
```

After we have the cross-compilation toolchain, how do we build the Linux Operating System for Raspberry pi 3?

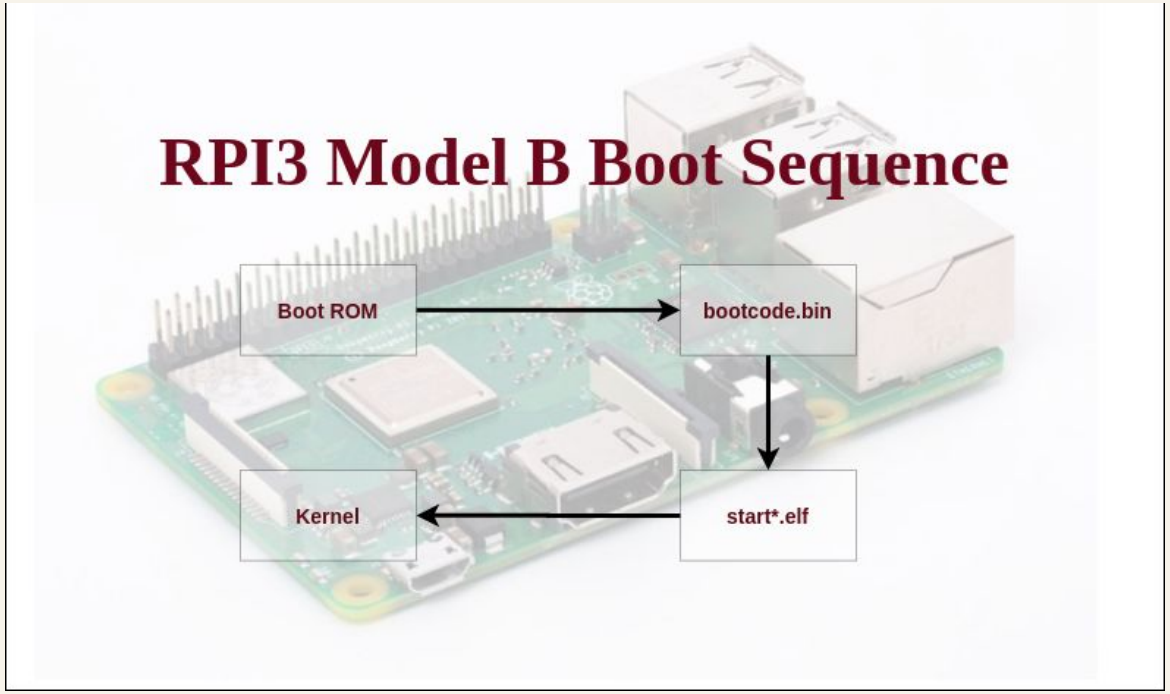
zhone
© 2026 Zhone – Proprietary and Confidential

03

Boot Process

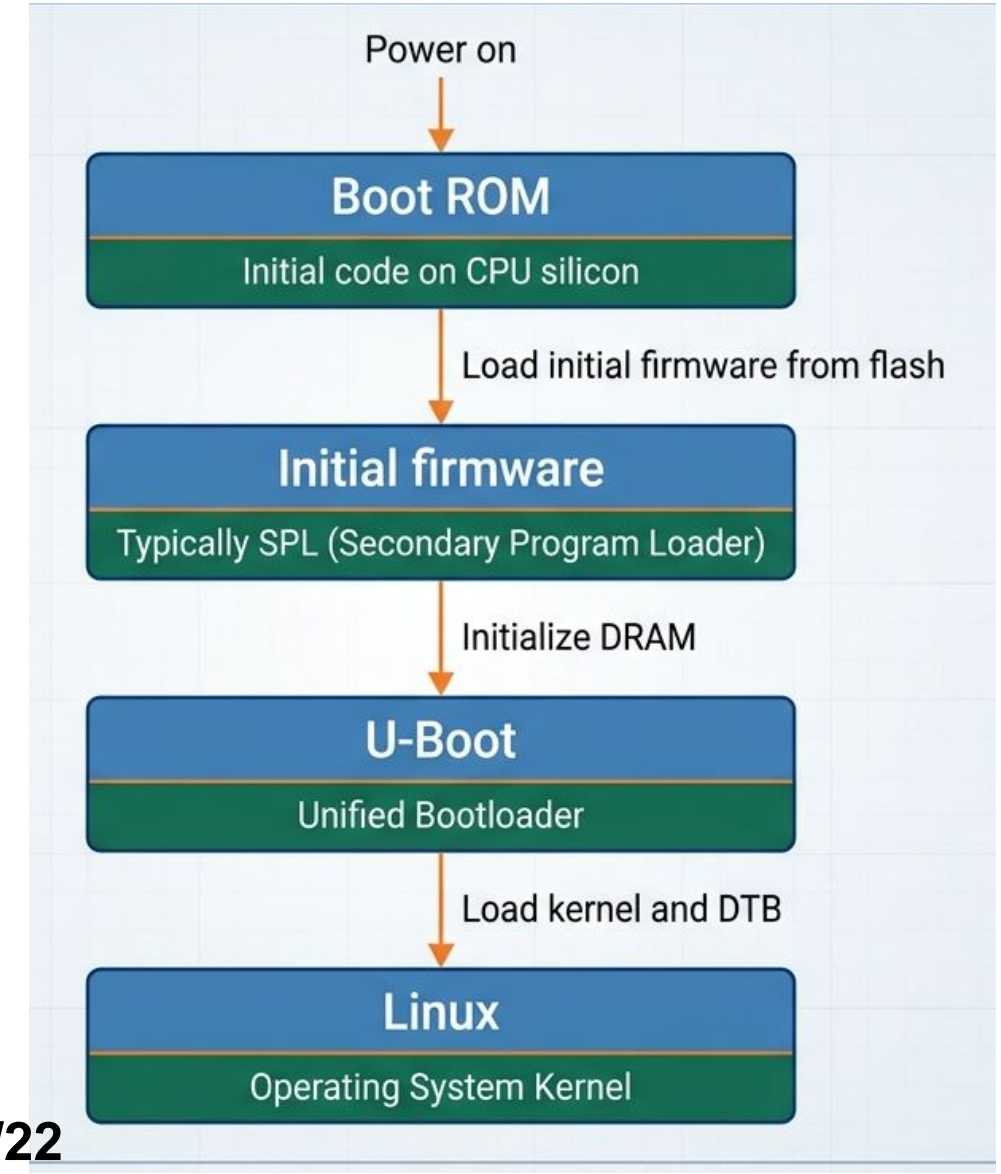
The boot process: How does the CPU go from power-on to the appearance of a Linux shell?

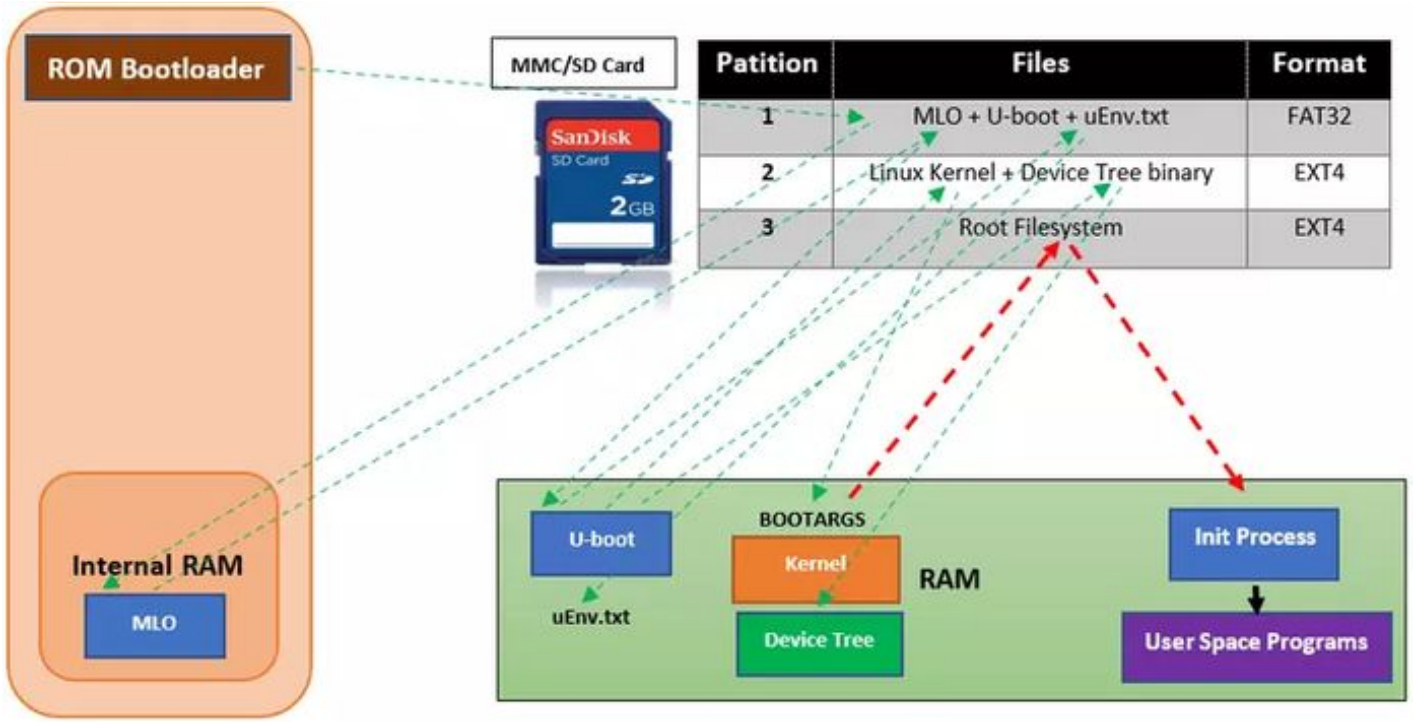
- The entire startup process.
- Linux kernel, Device Tree and root filesystem.
- What does U-Boot do?



zhone
© 2026 Zhone – Proprietary and Confidential

3. Boot Process





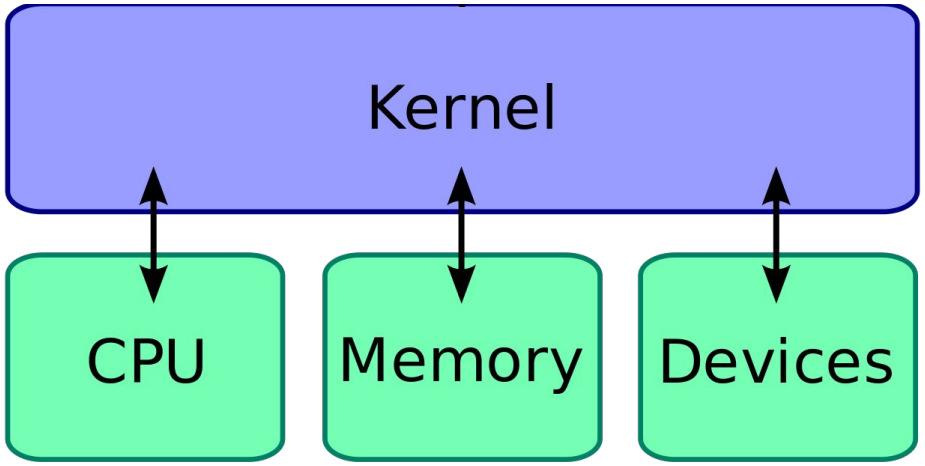
© 2026 Zhone – Proprietary and Confidential

3. Boot Process

Linux Kernel

What is the role of the kernel?

The Linux kernel is the core component of the operating system.



© 2026 Zhone – Proprietary and Confidential

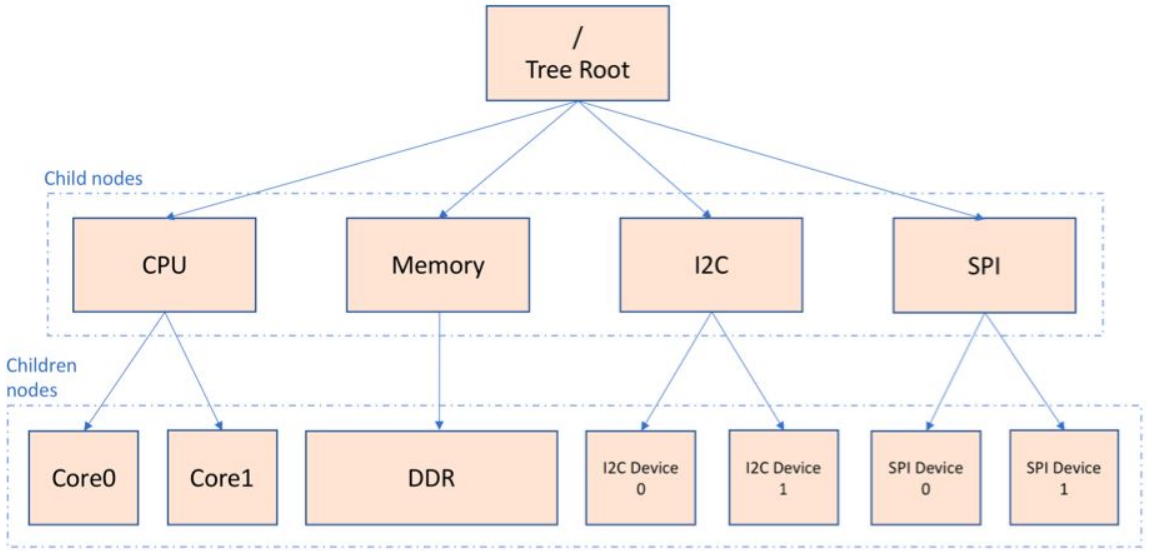
3. Boot Process

Device Tree

The kernel needs to know which hardware devices exist and how to access them.

A Device Tree describes information such as:

- ➔ Address ranges and hardware resources.
- ➔ UART, GPIO, Ethernet, and various buses.
- ➔ Declared peripheral devices.
- ➔ Information used by the kernel to select and initialize the appropriate drivers.



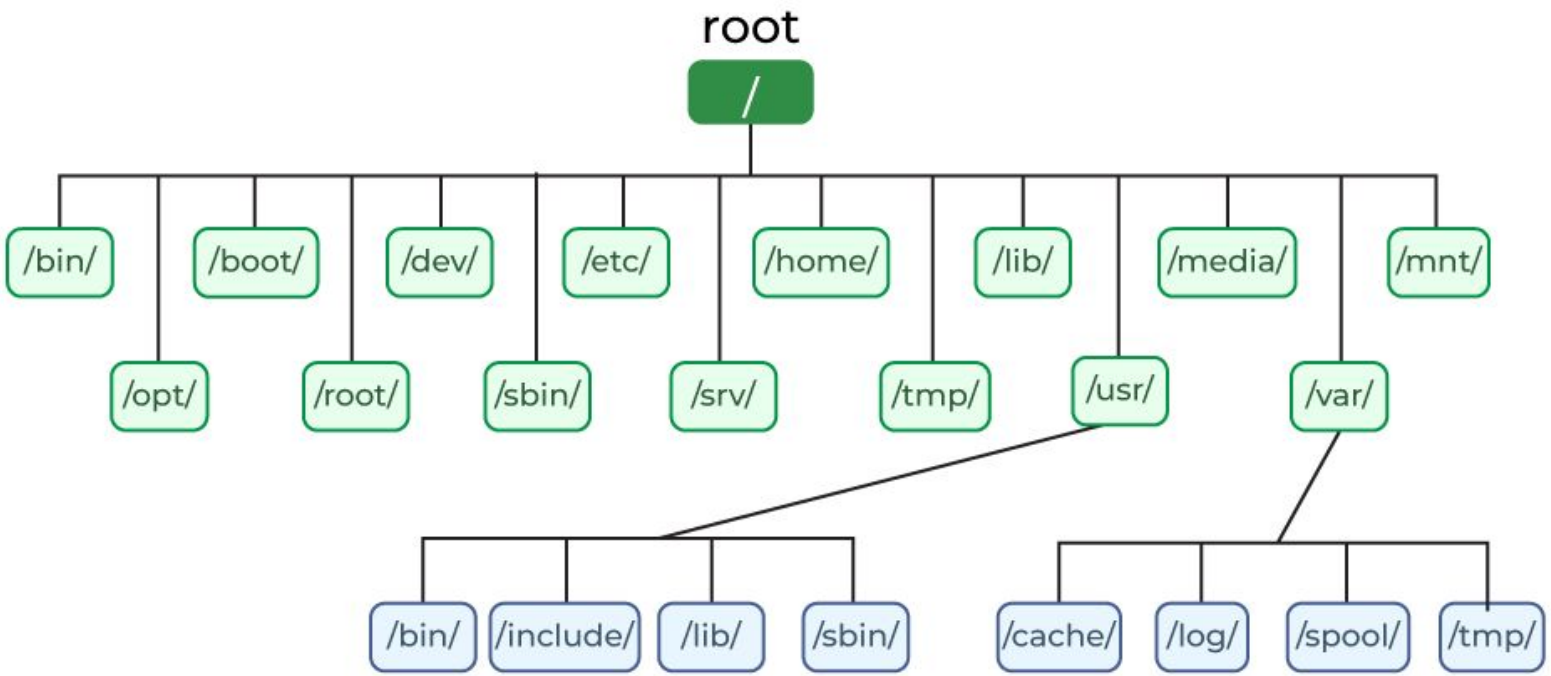
© 2026 Zhone – Proprietary and Confidential

3. Boot Process

Root Filesystem (rootfs)

What is a rootfs?

A root filesystem is the directory tree that the kernel uses as the root directory (/) after it has been successfully mounted. It contains the programs, libraries, configuration files, and components necessary to transform the kernel into a functional, usable system.



zhone
© 2026 Zhone – Proprietary and Confidential

Result the Simple Linux running on Raspberry pi 3 B

After building the U-Boot, kernel, device tree, and root filesystem using a cross-compilation toolchain, we can load these components onto an SD card and proceed with the boot process.

```
U-Boot 2025.10 (Oct 23 2026 - 21:59:21 +0700)

DRAM:  948 MiB
RPI 3 Model B (0xa02082)
Core:  88 devices, 13 uclasses, devicetree: board
MMC:   mmc@7e202000: 0, mmcnr@7e300000: 1
Loading Environment from FAT... Unable to read "uboot.env" from mmc0:1...
In:    serial,usbkbd
Out:   serial,vidconsole
Err:   serial,vidconsole
Net:   No ethernet found.

starting USB...
USB DWC2
Bus usb@7e980000: 4 USB Device(s)
       scanning usb for storage d
Hit any key to stop autoboot:  0
U-Boot>
U-Boot>
U-Boot>
```

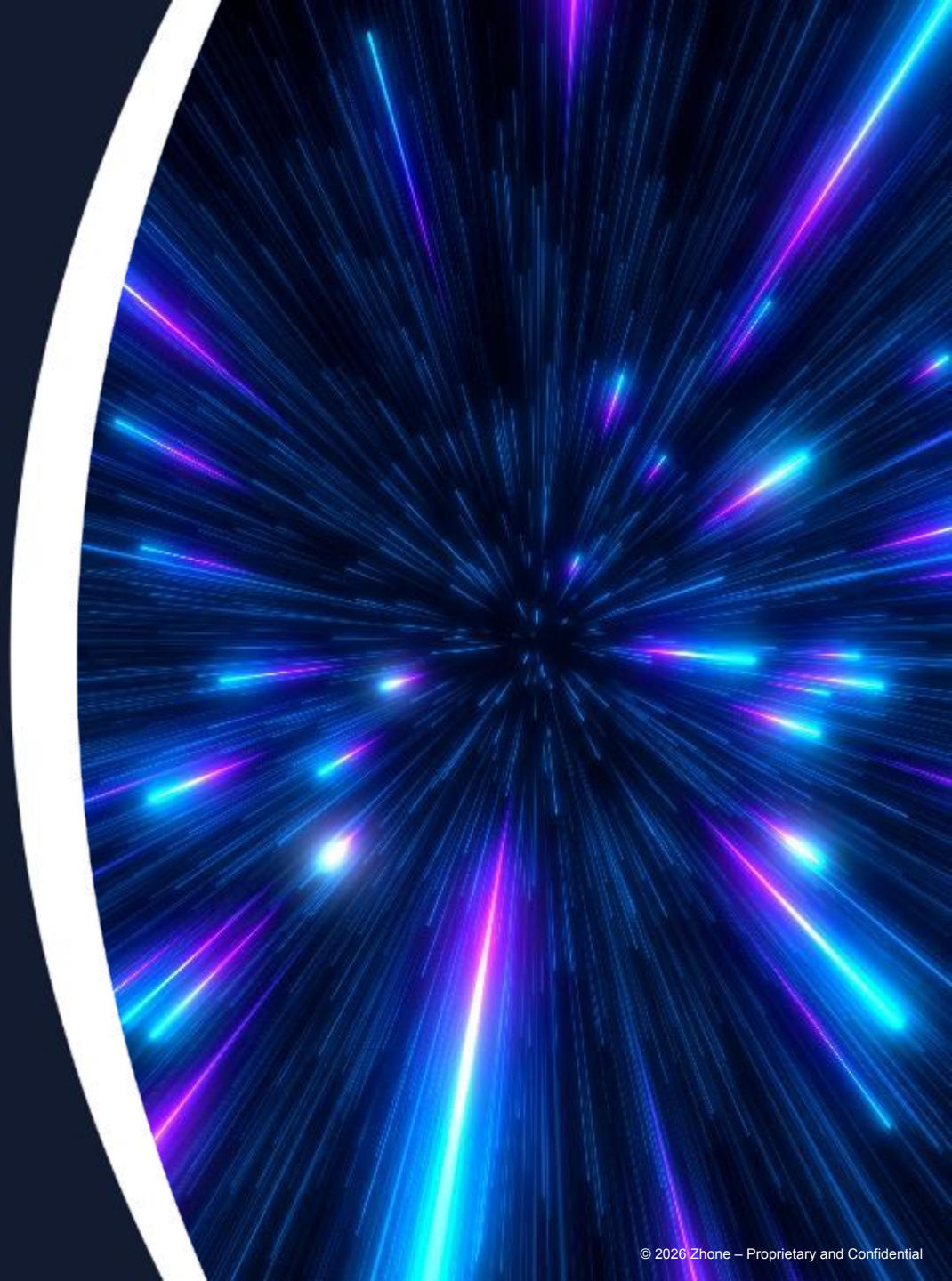
```
[init] Launching shell...
/bin/sh: can't access tty; job control turned off
~ # [    2.016865] hub 1-1:1.0: USB hub found
[    2.020213] hub 1-1:1.0: 5 ports detected
[    2.303840] usb 1-1.1: new high-speed USB device number 3 using dwc2
[   11.668506] platform wifi-pwrseq: deferred probe pending: pwrseq_simple: reset control not ready
[   11.676938] raspberrypi-power soc:power: sync_state() pending due to 3fc00000.v3d
[   11.684474] raspberrypi-power soc:power: sync_state() pending due to 3f902000.hdmi
[   11.692142] raspberrypi-power soc:power: sync_state() pending due to 3f806000.vec

~ # uname -a
Linux (none) 6.18.58 #1 SMP PREEMPT Fri Oct 23 23:18:57 +07 2026 aarch64 GNU/Linux
~ # cat /proc/cmdline
console=ttyS1,115200 earlycon root=/dev/mmcblk0p2 rootfstype=ext4 rootwait rw init=/init
~ #
```


© 2026 Zhone – Proprietary and Confidential

zhone

Thank you for listening...



Embedded Trainees

© 2026 Zhone – Proprietary and Confidential